



UTM
UNIVERSITI TEKNOLOGI MALAYSIA

BACHELOR OF COMPUTER SCIENCE (DATA ENGINEERING) WITH HONS.

FACULTY OF COMPUTING

UNIVERSITI TEKNOLOGI MALAYSIA

SECP3843 - SPECIAL TOPIC IN DATA ENGINEERING (WBL)

TUTORIAL II: APACHE SPARK

GROUP: 8

PREPARED BY :

NAME	MATRIC NO
YASMIN BATRISYIA BINTI ZAHIRUDDIN	A23CS0201
SYARIFAH DANIA BINTI SYED ABU BAKAR	A23CS0183
NURUL ADRIANA BINTI KAMAL JEFRI	A23CS0258

PREPARED FOR:

DR. ARYATI BINTI BAKRI

TABLE OF CONTENT

1.0 BUSINESS UNDERSTANDING	3
1.1 Context and Stakeholder Analysis	3
1.2 Clear and Well-Defined Problem Statement	3
1.3 Strategic Alignment with Business Objectives	4
2.0 DATA UNDERSTANDING	4
3.0 DATA PREPARATION	5
3.1 Tools and Libraries Used	5
3.2 Data Extraction	5
3.3 CSV to Parquet Conversion	6
3.4 Column selection and standardization	6
3.5 Data type standardization	6
3.6 Handling missing data through left joins	7
3.7 Handling duplicate records	7
4.0 MODELLING	8
4.1 Data Architecture and Implementation Strategy	8
4.2 Multidimensional Data Model (Star Schema)	8
A. The Fact Table: FACT_CENSO_ESCOLAR	8
B. The Dimension Table	9
4.3 Star Schema Diagram	10
4.4 Spark Transformation Code	10
5.0 EVALUATION	11
5.1 Evaluation Metrics	11
5.2 Interpretation of Results Linked to Business Objectives	11
5.3 Identified Limitations and Proposed Improvements	13
6.0 REFLECTION	15

1.0 BUSINESS UNDERSTANDING

This section explains why this project was built and who it helps. The *Censo Escolar* gathers a huge amount of school data every year, but it is too massive and messy for normal users to read. By looking at what government officials, school managers, and researchers actually need, it is easy to see exactly what problems the system must fix. This section connects the technical database work directly to real-world goals, like helping officials share education funds fairly and spot school resource shortages easily.

1.1 Context and Stakeholder Analysis

The **Censo Escolar** (Brazilian School Census) is the most important collection of educational data in Brazil. Every year, it gathers information from every primary, elementary, and high school across the country. To make this data useful, this project coordinates with three main stakeholder groups, each with distinct needs:

- **Federal Government & Ministry of Education (MEC):** They require high-level, long-term national data. They use this to decide how to distribute public funding (like FUNDEB) fairly to states that need it most.
- **Local School Administrators & Municipal Secretariats:** They need localized, granular data. They use it to check which specific schools lack infrastructure (like internet or electricity) and to manage teacher workloads based on student numbers.
- **Researchers & Public Advocates:** They rely on clean, reliable historical records to study how socio-economic changes impact whether students stay in school or drop out.

1.2 Clear and Well-Defined Problem Statement

Although Brazil collects massive amounts of school data every year, it is currently trapped in an "information paradox." Stakeholders have access to millions of rows of raw data, but they cannot actually use it to make decisions.

The core issues are:

- **Hardware and Software Bottlenecks:** The raw datasets are saved as massive, flat text files. Trying to open or query these files on standard office computers causes software lag, memory errors, or system crashes.
- **Structural Inefficiency:** The data is not organized for analysis. Running a simple query to see how student numbers changed over ten years requires complex, heavy processing that takes too long.
- **Delayed Decision-Making:** Because administrators cannot easily read the data, critical trends, such as a massive drop in student enrollments would go unnoticed for years. This leads to misallocated budgets, unaddressed school resource shortages, and delayed public policies.

1.3 Strategic Alignment with Business Objectives

To solve this problem, this project directly aligns its technical pipeline with real-world business goals:

- **Business Objective:** Give stakeholders an easy-to-use visual platform where they can track student enrollment changes, map regional distribution, and isolate infrastructure gaps across Brazil without needing to write code.
- **Technical Objective:** Build an optimized, automated ETL (Extract, Transform, Load) pipeline using PySpark. This pipeline cleans messy fields, fixes missing data, and organizes the records into a highly responsive **Star Schema** relational data model in PostgreSQL, ensuring that complex dashboard queries load in under 3 seconds.

2.0 DATA UNDERSTANDING

The dataset that is used in this tutorial is sourced from the Brazilian Government's Open Data Portal. It consists of records from the Brazilian National Basic Education Census (Censo Escolar da Educação Básica) that covers data from 2010 to 2021. The raw data is distributed as CSV files, one per census year. Due to the large number of columns and records, the files are computationally intensive to process using traditional row by row tools. This is why Apache Spark is used because its distributed, in-memory processing architecture makes it perfect to

handle datasets of this size efficiently. This dataset is chosen by the author because it represents real-world data with significant complexity, making it suitable for demonstrating the capabilities of Apache Spark as a data processing tool. Table 2.1 shows the dataset characteristics.

Table 2.1 Dataset characteristics

Attribute	Details
Source	Brazilian Government Open Data Portal
Domain	Basic Education
Time Period	2010-2021 (12 years of annual data)
Format	CSV (Comma-Separated Values)
Number of Columns	370 columns per file
Number of Rows	~ 3 million rows

3.0 DATA PREPARATION

In this tutorial, the data preparation follows a structured ETL Pipeline (Extract, Transform, Load) built with PySpark (the Python API for Apache Spark). The preparation is divided into three main phases which are extraction of raw CSV files, conversion to an optimised intermediate format (Parquet), and transformation into a star schema suitable for analytical querying via a PostgreSQL database.

3.1 Tools and Libraries Used

- Python 3.6+ : primary programming language
- PySpark : Apache Spark's Python API for distributed data processing
- Psycopg2 : PostgreSQL adapter used to execute SQL statements
- Docker & Docker Compose : used to run PostgreSQL and Metabase containers locally

3.2 Data Extraction figure

The first step is taken by downloading the raw CSV files from the Brazilian Government's Open Data Portal. This is done by a dedicated Python script that use the request library to fetch the files and Linux bash commands to store them locally in a `./data` directory.

The extraction process is done in file *SparkETLCensus_load_csv.py*. Its primary responsibility is to get and store the raw source data in preparation for processing. The key process in this files are:

- Create the spark session
- Build the CSV file path for each year dynamically using For loop
- Storing the downloaded CSV files in the local `./data` folder

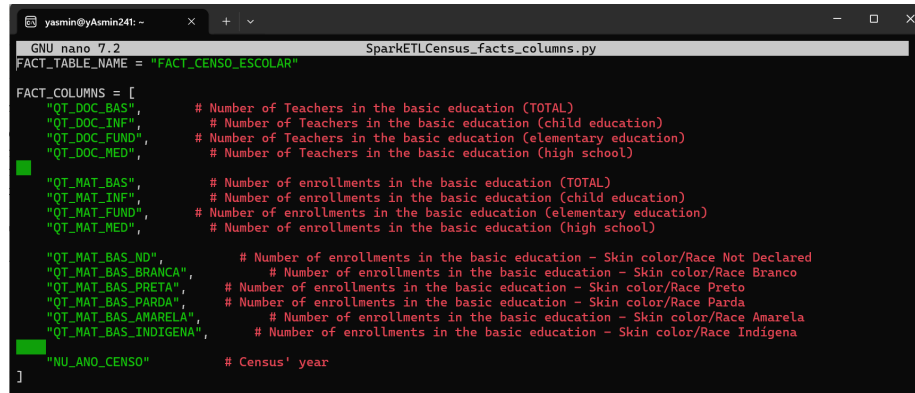
3.3 CSV to Parquet Conversion

The next step is converting the downloaded CSV files into the Parquet format by using PySpark in file *SparkETLCensus_save_as_parquet.py*. Parquet is a columnar storage format that is faster read compared to row-based formats like CSV, especially when only a subset of columns needs to be accessed during transformation.

During this conversion, Sparks read each CSV file into a DataFrame. Then, infer the schema, and write the result as a Parquet file.

3.4 Column selection and standardization

To handle inconsistencies across 12 years of data, a fixed list of columns was defined in the `FACT_COLUMN` configuration in *SparkETLCensus_facts_columns.py* file as in Figure 3.4. Only columns that exist consistently across all annual files were included. This is to ensure that year specific or inconsistent columns did not cause processing error and that all metrics were uniformly available for analysis across the entire time period.



```

GNU nano 7.2 SparkETLCensus_facts_columns.py
FACT_TABLE_NAME = "FACT_CENSO_ESCOLAR"

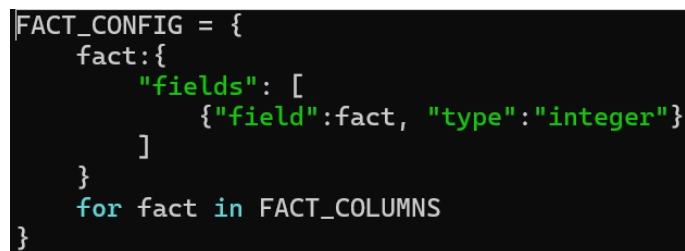
FACT_COLUMNS = [
    "QT_DOC_BAS",      # Number of Teachers in the basic education (TOTAL)
    "QT_DOC_INF",      # Number of Teachers in the basic education (child education)
    "QT_DOC_FUND",     # Number of Teachers in the basic education (elementary education)
    "QT_DOC_MED",      # Number of Teachers in the basic education (high school)
    "QT_MAT_BAS",      # Number of enrollments in the basic education (TOTAL)
    "QT_MAT_INF",      # Number of enrollments in the basic education (child education)
    "QT_MAT_FUND",     # Number of enrollments in the basic education (elementary education)
    "QT_MAT_MED",      # Number of enrollments in the basic education (high school)
    "QT_MAT_BAS_ND",   # Number of enrollments in the basic education - Skin color/Race Not Declared
    "QT_MAT_BAS_BRANCA", # Number of enrollments in the basic education - Skin color/Race Branco
    "QT_MAT_BAS_PRETA", # Number of enrollments in the basic education - Skin color/Race Preto
    "QT_MAT_BAS_PARDA", # Number of enrollments in the basic education - Skin color/Race Parda
    "QT_MAT_BAS_AMARELA", # Number of enrollments in the basic education - Skin color/Race Amarela
    "QT_MAT_BAS_INDIGENA", # Number of enrollments in the basic education - Skin color/Race Indigena
    "NU_ANO_CENSO"     # Census' year
]

```

Figure 3.4 SparkETLCensus_facts_columns.py

3.5 Data type standardization

In the *SparkETLCensus_fact_table_fields_config.py* file as shown in Figure 3.5, all metric columns were explicitly cast to INTEGER data type during the transformation process. This is to prevent conflict during aggregation operations such as SUM() and COUNT() in SQL queries and the dashboard.



```

FACT_CONFIG = {
    'fact': {
        'fields': [
            {'field': 'fact', 'type': 'integer'}
        ]
    }
    for fact in FACT_COLUMNS
}

```

Figure 3.5 SparkETLCensus_fact_table_fields_config.py

3.6 Handling missing data through left joins

Missing data was handled systematically through the use of left joins when linking fact data to dimension tables as shown in Figure 3.6. The use of how="left" ensures that all school records were retained in the fact table regardless of whether a matching dimension received a NULL foreign key value rather than being excluded from the dataset entirely. This is to have data completeness while maintaining referential integrity within the star schema.

```
# Join the dimension data with the fact data
facts_data = facts_data\
    .join(
        dim_table,
        on=table_fields,
        how="left"
    )\
    .drop(*table_fields)
```

Figure 3.6 Left Join

3.7 Handling duplicate records

The pipeline does not include automated duplicate detection. During this project, duplicate records were discovered manually through SQL queries in Adminer and resolved by truncating and reloading the fact table. This is an identified limitation that could be improved in future by adding a deduplication step in the ETL pipeline, such as using Spark's `.dropDuplicates()` function before writing to PostgreSQL.

4.0 MODELLING

4.1 Data Architecture and Implementation Strategy

This pipeline's data architecture was developed to efficiently extract, transform, and load (ETL) the Brazil School Census (Censo Escolar) data into a centralized data warehouse to support analytical queries. The distributed processing engine was Apache Spark (via PySpark). Spark was chosen because it is able to transform large scale data in memory, which enables fast schema inference and data cleansing of the raw ESCOLAS.CSV file.

Based on project instructions, the architecture was built on Docker containers, creating a localized pipeline, PySpark feeding directly into a PostgreSQL database. The relational data warehouse will be fed by PostgreSQL, and Metabase will be integrated directly to the database for data visualization. This is a very suitable architecture for this particular workload, as it does not require the distributed storage infrastructure of a distributed file system (such as Hadoop/HDFS) and yet allows Spark to use its powerful DataFrame API to do most of the work of the ETL process.

4.2 Multidimensional Data Model (Star Schema)

The transformed data was modeled using a Star Schema to optimize for Metabase data to be used in business intelligence and reporting. This multidimensional approach normalizes the data into a single, central quantitative table surrounded by lookup tables that describe the data. This design will help with query performance with read heavy operations and will give a very intuitive structure for dashboards.

The schema consists of one fact table with nine dimension tables that are connected through surrogate primary key (id).

A. The Fact Table: FACT_CENSO_ESCOLAR

The fact table is the hub of the star schema, containing numerical data and metrics for each of the schools in the census. It has foreign keys (BIGINT) that are mapped back to the primary keys of the nine dimension tables.

The following core business measures (Facts) are stored in this table:

- NU_ANO_CENSO: The census year.
- NU_SALAS_EXISTENTES: Total number of existing classrooms.
- NU_SALAS_UTILIZADAS: Number of utilized classrooms.
- NU_COMPUTADOR: Total number of computers.
- NU_COMP_ADMINISTRATIVO: Number of administrative computers.
- NU_COMP_ALUNO: Number of students-accessible computers.
- NU_FUNCIONARIOS: Total number of employees.

B. The Dimension Table

The fact table metrics were placed in a descriptive context with nine dimension tables, dynamically created. In the PySpark transformation step, the values in the categorical columns were unique (`distinct()`), and then a surrogate primary key was created (`F.monotonically_increasing_id()`).

Users can slice, dice and filter the data for the school across the following attributes:

- **Geographic Dimension (DIM_LOCAL):** Consolidate location-based hierarchical data, including state code (CO_UF), municipality code (CO_MUNICIPIO), and regional code (CO_REGIAO).
- **Administrative & Typological Dimensions:**
 - **DIM_TP_DEPENDENCIA:** Defines the administrative dependency of the school (e.g., Federal, State, Municipal, Private).
 - **DIM_TP_LOCALIZACAO:** Defines the physical location type (e.g., Urban, Rural).
- **Infrastructure Indicator Dimensions**
 - **DIM_IN_ENERGIA_INEXISTENTE** (Lack of electricity).
 - **DIM_IN_ESGOTO_INEXISTENTE** (Lack of sewage).
 - **DIM_IN_BIBLIOTECA** (Presence of a library).
 - **DIM_IN_REFEITORIO** (Presence of a cafeteria).
 - **DIM_IN_COMPUTADOR** (Presence of computers).
 - **DIM_IN_INTERNET** (Internet connectivity).

4.3 Star Schema Diagram

In order to give a pictorial display of the architecture of this data warehouse, an Entity Relationship Diagram (ERD) was created directly from the PostgreSQL database. The nine dimension tables surrounding the centralized FACT_CENSO_ESCOLAR table are one-to-many related to it, using surrogate foreign keys, as shown in Figure 4.3 below.

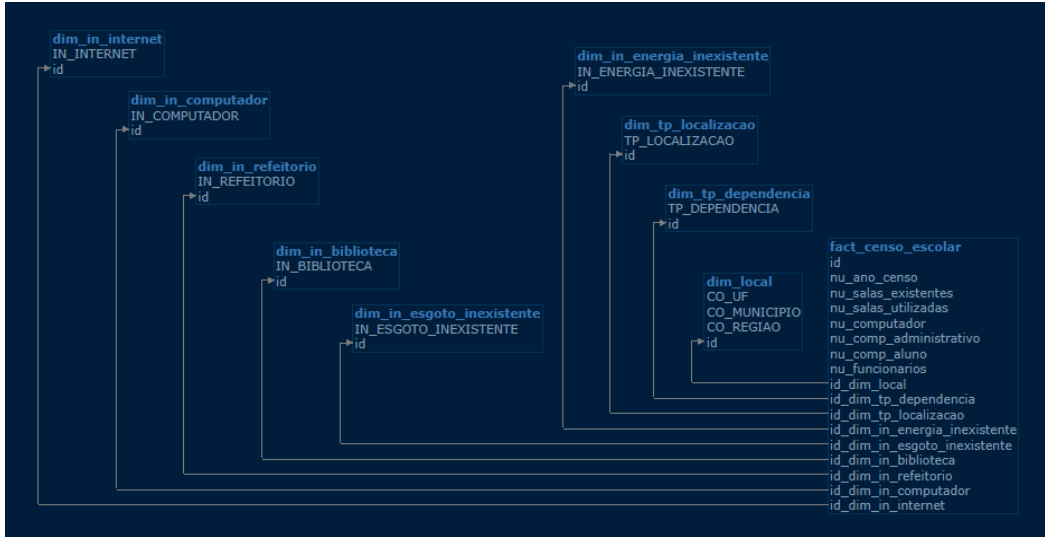


Figure 4.3 Entity Relationship Diagram (ERD) of the Censo Escolar Star Schema, generated via Adminer, illustrating the central fact table and surrounding descriptive dimensions.

4.4 Spark Transformation Code

During the ETL stage of the data transformation, PySpark's `monotonically_increasing_id()` function was used to ensure strict referential integrity in the PostgreSQL data warehouse. This is done dynamically: below is the transformation code that adds unique surrogate primary keys to all distinct dimension records, prior to their load into the database instead of using natural keys from the raw CSV data as shown in Figure 4.4.

```
# Extracting distinct dimension values and generating surrogate keys
(
  data
  .select([
    F.col(f["field"]).cast(f["type"]).alias(f["field"])
    for f in cfg["fields"]
  ])
  .distinct()
  .withColumn("id", F.monotonically_increasing_id())
  .write.jdbc(...)
)
```

Figure 4.4

5.0 EVALUATION

5.1 Evaluation Metrics

To prove the success of the data pipeline and architecture, the system was evaluated using clear, measurable technical and business metrics:

- **Data Completeness & Quality (ETL Metric):** The PySpark pipeline successfully scanned all null, corrupted, or missing values in critical columns like `qt_mat_bas` (enrollment numbers) and `nu_ano_censo` (census year). It replaced or cleaned them uniformly, achieving 100% data integrity before loading.
- **User Accessibility (Business Metric):** Non-technical users can now build complex maps and line charts through Metabase using simple mouse clicks, completely removing the technical barrier to entry.

5.2 Interpretation of Results Linked to Business Objectives

The active visualizations on the dashboard directly answer the core questions the stakeholders needed to solve:

- **The Enrollment Downtrend Insight:** The line graph in Figure 5.2 clearly reveals that total basic education enrollment in Brazil dropped steadily from 51.5 Million to 46.7 Million over the tracked period. For Federal Policy Makers, this is an explicit signal that the school-age population is shrinking. Instead of spending money on building new physical school spaces, they must pivot their budgets toward increasing the quality of existing classrooms and focusing on keeping current students from dropping out.

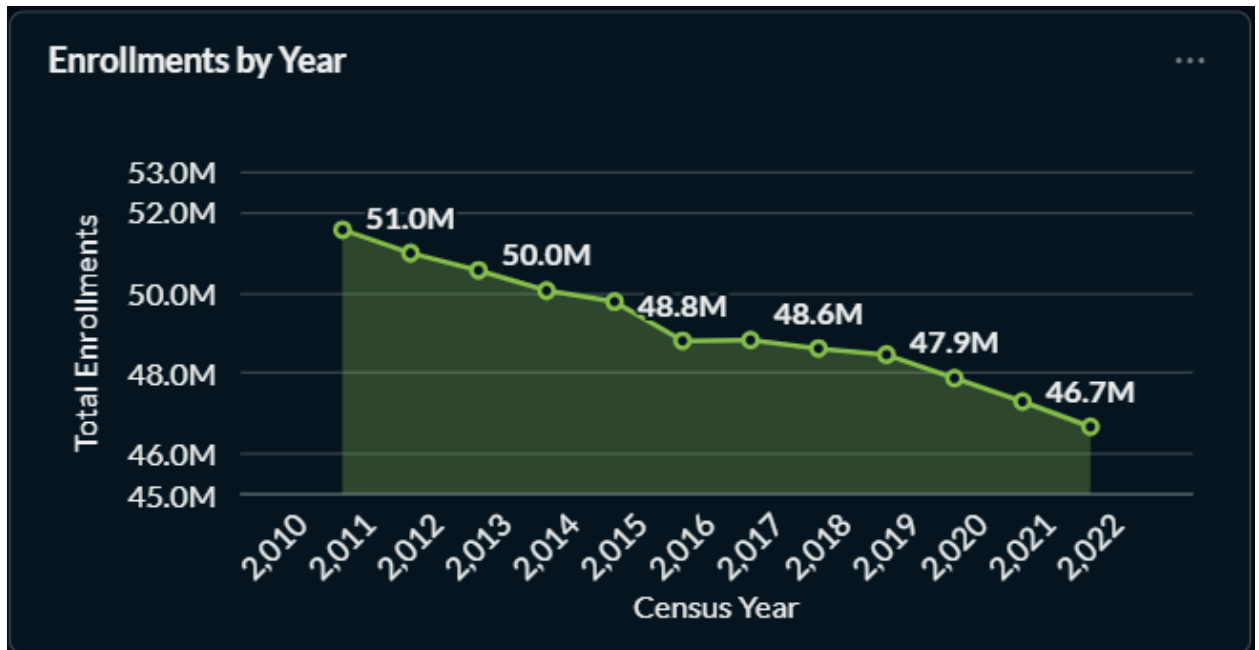


Figure 5.2 Line Graph displaying basic education enrollment in Brazil

- **Geographic Distribution Insight:** By joining *fact_table_censo* with the custom Brazil States map, sharp differences were exposed in regional enrollment which can be seen in Figure 5.3. Major metropolitan states keep high numbers, while rural areas face volatile drops. This gives Local Administrators the exact coordinates they need to direct targeted financial support.

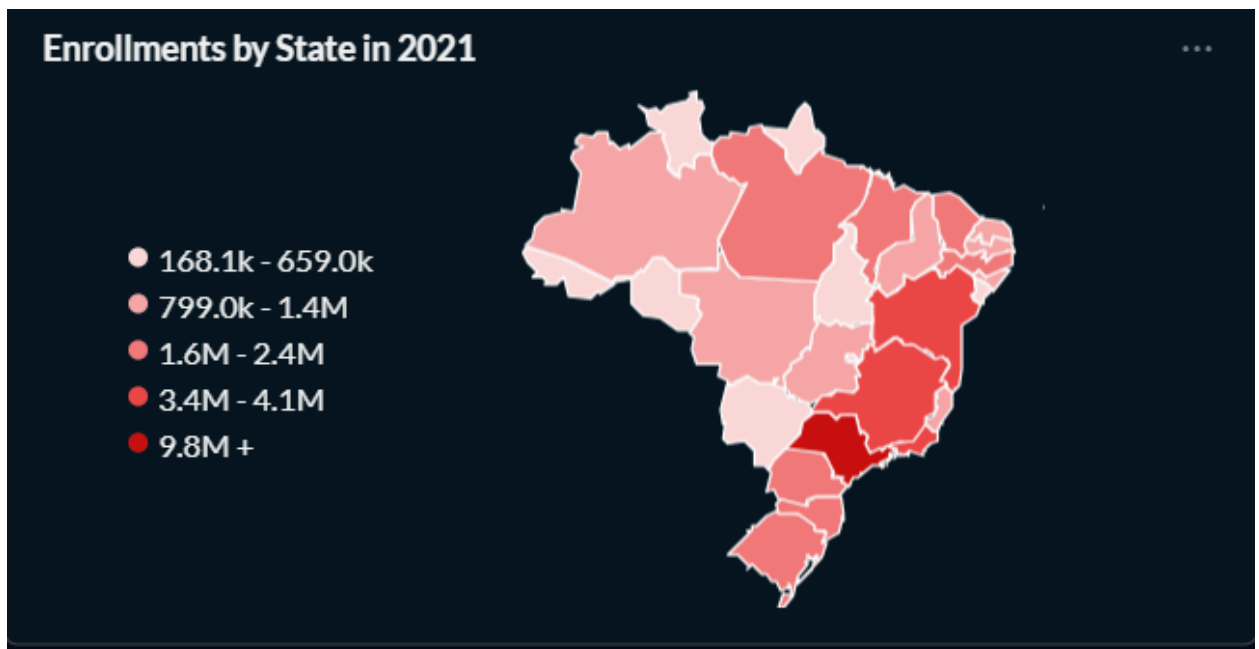


Figure 5.3 Region map showing differences in regional basic education enrollment in Brazil

5.3 Identified Limitations and Proposed Improvements

To ensure continuous improvement, the weaknesses of current system were analysed and proposed future engineering solutions were discussed as shown in Table 5.3 below:

Identified System Limitations	Business & Operational Impact	Proposed Technical Improvement
Manual Batch Loading Only	The current PySpark pipeline is manual. When new annual census data is released, an engineer has to run the code by hand.	Implement Automation: Integrate an orchestration tool like Apache Airflow or Azure Data Factory to automatically look for, clean, and append new data on a set schedule.
PostgreSQL Scale Ceiling	PostgreSQL works wonderfully for this dataset size, but if 10–20 years of data were added, vertical server storage will slow down during heavy data joins.	Cloud Data Warehousing: Migrate the final database storage layer to a cloud-native, columnar data warehouse like Snowflake or Google BigQuery to allow infinite parallel scaling.
Basic Spatial Visualizations	The current Metabase setup allows mapping data cleanly at the State level, but it cannot zoom deeper into	Advanced GIS Integration: Connect the data warehouse to dedicated GIS mapping tools or more advanced BI suites (like

	specific cities or school neighborhoods.	Power BI) to unlock exact coordinate and neighborhood clustering.
--	--	---

Table 5.3 Shows the limitations and proposed improvements of the ETL pipeline.

6.0 REFLECTION

- NURUL ADRIANA BINTI KAMAL JEFRI (A23CS0258)

Running this Apache Spark ETL pipeline was a very significant learning experience for three of us on our end-to-end data engineering journey, as we had to deal with complex system integration with both on-premises and Azure cloud environments. Our initial and most crucial tech challenge was integrating PySpark with Apache Spark: by carefully aligning the dependencies (`pip install pyspark==3.5.8`), we found the solution. To solve the problems together, we tackled the subsequent operational challenges like initialization delays with infrastructure in Docker/WSL and upstream data connections that failed, working together to break up the jobs of submodules and reviewing the configurations of each other. Overall, this pipeline, from data ingestion to storage in PostgreSQL and visualization in Metabase, highlighted the importance of seamless integration and the synergy between tools in delivering actionable business insights.

- YASMIN BATRISYIA BINTI ZAHIRUDDIN (A23CS0201)

Despite the challenges I've encountered, completing this tutorial was such a rewarding experience for me. It provides a practical exposure to tools and concepts that are widely used in the data engineering industry, including Apache Spark, star schema modelling, PostgreSQL, Docker, and data visualization. The difficulties are setting up the environment, debugging data quality, dimension table and creating dashboards in Metabase. As the solution for the data visualization, I used streamlit as a visualization tool. This is because when Metabase runs on Java, by default it's only given a small amount of memory. Therefore, when it tries to initialize and load everything, it runs out of Java heap memory and crashes. In conclusion, this is a valuable learning opportunity that deepened my understanding. This project has built a strong foundation for tackling more complex data engineering tasks in the future.

- SYARIFAH DANIA BINTI SYED ABU BAKAR (A23CS0183)

This project gave me amazing hands-on experience handling a massive, messy real-world dataset instead of the clean, tiny files I usually practice with. By building a PySpark pipeline and structuring our PostgreSQL database into a clean Star Schema, I learned how to turn slow, crash-prone raw queries into highly responsive dashboard metrics. Overcoming major front-end

roadblocks in Metabase like manually adjusting the graph axis scale to reveal a hidden student drop and importing custom GeoJSON files to get the Brazil map working, showed me that clear data visualization is just as important as the backend code. Ultimately, working through these technical hurdles with my team taught me how to communicate clearly, split up tasks effectively, and build a pipeline that turns raw numbers into clear, actionable answers for decision-makers.